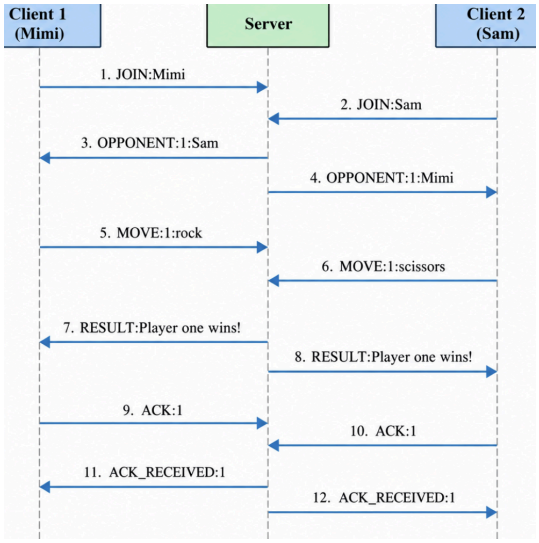
Writeup:

- Protocol: My program uses UDP datagrams to communicate between a central server and two clients. The server assigns players, receives moves, determines the winner, and manages the game state.
    - Message Reference Table:

Message	Sender	Fields / meaning
`JOIN`	Client	Player name; requests registration
`OPPONENT`	Server	Game ID and opponent name; confirms pairing
`MOVE`	Client	Game ID and selected move
`WAIT`	Server	Status message indicating that an opponent is needed
`ERROR`	Server	Error description, such as an invalid move
`RESULT`	Server	Winner or tie announcement
`ACK`	Client	`RESULT` ; acknowledges receipt
`ACK_RECEIVED`	Server	`RESULT` ; confirms the acknowledgment

    - Completed game timeline:



- Reliability:
    - Timeout: The client socket uses setSoTimeout(2000), meaning a two-second receive timeout.
    - Retransmission: When the timeout expires, receiveWithRetry() resends the message associated with the current phase.

- Duplicate messages: The server recognizes previously registered players and avoids replacing an already stored move.
- Lost results: The server retains lastResult and can resend it in response to a repeated move.
- Acknowledgments: Both players send ACK:RESULT, and the server replies with ACK_RECEIVED:RESULT.
- Delayed and out-of-order packets: Game IDs on MOVE messages allow the server to reject moves for the wrong game.

- Testing:
  -

Test:	Observations:
Normal UDP game	Both clients received the result
Rock, paper, scissors outcomes	Winning and tie outcomes were tested
Lost JOIN packets	Repeated JOINs eventually reached the server
Lost MOVE or RESULT packets	Retransmissions allowed the game to proceed
Lost ACK messages	Server confirmation was received after retries
Consecutive games	Server reset and accepted new players
20% loss, 20% delay, 1000 ms	Successful games after resolving earlier problems

  - I first tested the program locally in Eclipse using one server and two clients, and checked that both players could join, receive their opponent's name, submit a move, and receive the correct result. I also tested different combinations of rock, paper, and scissors, including ties.
  - I used lossy_proxy.py with a separate proxy for each client. Test settings:

        - Packet loss: 20%
        - Packet delay: 20%
        - Maximum delay: 1000 ms
        - Server port: 9000
        - Proxy ports: 9001 and 9002
    - These tests checked whether the game could recover when UDP packets were dropped or delayed.
    - My final tests successfully completed games under normal conditions and with the lossy proxy configured for 20% loss and 20% delay. I also tested the server's ability to reset and accept another pair of players.
- AI use: AI use on this project included debugging and testing guidance, and all written codes and words are my own.